-THE ROLE OF A PARSER

In this process of compilation, the parser and lexical analyzer work together. That means, when parser required string of tokens it invokes lexical analyzer produce string of token. In turn, the lexical analyzer supplies tokens to syntax analyzer (parser).

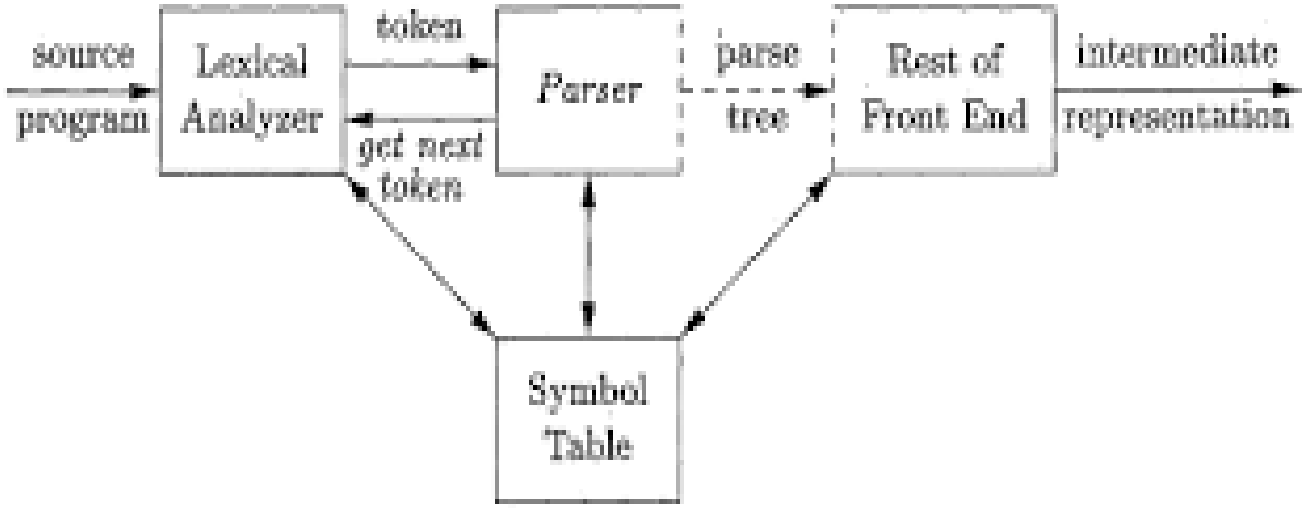


Figure 4.1: Position of parser in compiler model

The parser collects sufficient number of tokens and builds a parse tree. Then by building the parse tree, parse smartly finds the syntactical errors if any. It is also necessary that the parse should recover from commonly occurring errors so that remaining task of process the input can be continued.

The role of the parser code source tokens errors scanner parser IR Parser

- performs context-free syntax analysis
- guides context-sensitive analysis
- constructs an intermediate representation
- produces meaningful error messages
- attempts error correction

In the compiler model, the parser obtains a string of tokens from the lexical analyser, and verifies that the string can be generated by the grammar for the source language.

- The parser returns any syntax error for the source language.

- There are three general types' parsers for grammars.
- Universal parsing methods such as the *Cocke-Younger-Kasami algorithm* and *Earley's algorithm* can parse any grammar. These methods are too inefficient to use in production compilers.

The methods commonly used in compilers are classified as either *top-down parsing* or *bottom-up parsing*.

1. *Top-down parsers* build parse trees from the *top (root)* to the *bottom (leaves)*.
2. *Bottom-up parsers* build parse trees from the *leaves* and work up to the *root*.

- In both case input to the parser is scanned from left to right, one symbol at a time.
- The output of the parser is some representation of the parse tree for the stream of tokens.

There are number of tasks that might be conducted during parsing. Such as;

1. Collecting information about various tokens into the symbol table.
2. Performing type checking and other kinds of semantic analysis.

3. Generating intermediate code.

Syntax Error Handling:

1. Planning the error handling right from the start can both simplify the structure of a compiler and improve its response to errors.
2. The program can contain errors at many different levels. e.g.,
   a. *Lexical* – such as misspelling an identifier, keyword, or operator.
   b. *Syntax* – such as an arithmetic expression with unbalanced parenthesis.
   c. *Semantic* – such as an operator applied to an incompatible operand.
   d. *Logical* – such as an infinitely recursive call.
   e. Much of the error detection and recovery in a compiler is centered on the syntax analysis phase.
   f. One reason for this is that many errors are syntactic in nature or are exposed when the stream of tokens coming from the lexical analyser disobeys the grammatical rules defining the programming language.

Another is the precision of modern parsing methods; they can detect the presence of syntactic errors in programs very efficiently.

i. The error handler in a parser has simple goals:
ii. It should the presence of errors clearly and accurately.
iii. It should recover from each error quickly enough to be able to detect subsequent errors.
iv. It should not significantly slow down the processing of correct programs.

Error-Recovery Strategies:

- There are many different general strategies that a parser can employ to recover from a syntactic error.
  1. Panic mode
  2. Phrase level
  3. Error production
  4. Global correction
- *Panic mode:*
  - This is used by most parsing methods.
  - On discovering an error, the parser discards input symbols one at a time until one of a designated set of synchronizing tokens (delimiters; such as; semicolon or end) is found.
  - Panic mode correction often skips a considerable amount of input without checking it for additional errors.
  - It is simple.
- *Phrase-level recovery:*
  - On discovering an error; the parser may perform local correction on the remaining input; i.e., it may replace a prefix of the remaining input by some string that allows the parser to continue.
  - e.g., local correction would be to replace a comma by a semicolon, deleting an extraneous semicolon, or insert a missing semicolon.
  - Its major drawback is the difficulty it has in coping with situations in which the actual error has occurred before the point of detection.
- *Error productions:*
  - If an error production is used by the parser, can generate appropriate error diagnostics to indicate the erroneous construct that has been recognized in the input.

- *Global correction:*
  - Given an incorrect input string x and grammar G, the algorithm will find a parse tree for a related string y, such that the number of insertions, deletions and changes of tokens required to transform x into y is as small as possible.

Context-Free Grammars.

A **context-free grammar** (CFG) is a set of recursive rewriting rules (or productions) used to generate patterns of strings. A CFG consists of the following components:

- a set of terminal symbols, which are the characters of the alphabet that appear in the strings generated by the **grammar**
- a set of *nonterminal symbols*, which are placeholders for patterns of terminal symbols that can be generated by the nonterminal symbols.
- a set of *productions*, which are rules for replacing (or rewriting) nonterminal symbols (on the left side of the production) in a string with other nonterminal or terminal symbols (on the right side of the production).
- a *start symbol*, which is a special nonterminal symbol that appears in the initial string generated by the grammar.

To generate a string of terminal symbols from a CFG, we:

- Begin with a string consisting of the start symbol;
- Apply one of the productions with the start symbol on the left hand size, replacing the start symbol with the right hand side of the production;
- Repeat the process of selecting nonterminal symbols in the string, and replacing them with the right hand side of some corresponding production, until all nonterminals have been replaced by terminal symbols.

For example: **Simple Arithmetic Expression**

- An integer is an arithmetic expression
- If expression 1 and expression 2 are arithmetic expression, the so are following:

  Expr 1 – Expr 2
  Expr 1 / Expr 2
  (Expr1)

The corresponding CFG:

Expr --→ INITLITERAL	E -→ initlit
Expr --→ Expr MINUS Expr	E--→ E – E
Expr --→ Expr DIVIDE Expr	E-→ E / E
Expr – LPAREN Expr RPAREN	E-→(E)

A more compact way to write above grammar:

E--→ initlit
| E – E
| E / E
| (E)

Simplification of CFG

Simplification of CFG, is important to simplify because simplify grammar easy to process. To simplifying CFG there are 3 steps:

1. Removal of Null Variable
2. Removal of Unit Production
3. Removal of Useless Variable

1. **Removal of Nullable Variable**

   Any production rule that is terminated by **λ or ε is** classified as Nullable variable. E.g. a→ε

Prof(Dr) Anil Kumar

To simplify grammar, we need to eliminate them.
S→ aMb
M→ aMb
M→ λ
In the above grammar M→ **λ, is** a Nullable variable, therefore eliminate it
S→aMb [**Replace M by λ] →**
Λ
Since M containing **λ production, so** replace M by **λ,** everywhere and after replacing, add that production to grammar without **λ.**
S→aMb | ab
M→aMb | ab

2. **Removal of Unit Production**
any production, where both side single variable [Capital Letter], such as A→B is known as Unit Production.
e.g. A→A [**if any production has unit production, eliminate it immediately from the grammar].**
For example:
S→aA
A→a
A→B
B→A
B→b
In above grammar A→B is Unit production, eliminate from grammar & substitute A→B;
Replace A & B, wherever it is possible in production, because it is Unit Production.
Classified grammar is
S→aA | aB
A→a
New Grammar:
S→aA|aB
A→a
B→A|B
B→ bb
In the above grammar B→B and B→A is unit production.
Remove B→B, every production from B→B should be removed immediately.
S→aA|aB
A→a
B→A
B→bb
From the above grammar eliminate next unit production i.e., B→A and substitute B→A [**A→a, B→A ➔ aA]**
After this we get this grammar
S→aA|aB|aA
A→a
B→bb
In first production aA are two times, so remove repeated production
Finally grammar is
S→aA|aB
A→a
B→bb

3. **Removal of useless variable**
If production directly generating a string terminal, then that production is useful.

Prof(Dr) Anil Kumar

But if production, after a number of steps, it not able to generate string that production is useless. Therefore, it needs to eliminate from grammar.

For example:

S→a|b [Production directly generating the strings, so it is not useless]

A→B[There is loop going on, you not able to generate any string out of this,

B→C [therefore these are useless variable, so need to eliminate]

C→ A

For Example

S→aS|A|C

A→a

B→aa

C→ aCb

Find all variable, that can be produce string with only terminal. Here, we can see, A & B are only production that will generate a string terminal.

So make a set **{A, B}** of it.

Now each production, check that if there is any combination of A & B be present.

We find that it is first production **S→A, so add it in the set {A, B, S}**

Keep only those variable, that produce terminal symbol {A, B, S}, rest of variable is useless.

In the above grammar, we can see that C is only variable, i.e. not present in set. Hence it is useless variable, so eliminate it.

S→aS|A

A→a

B→aa [**B is not connected to first Production, so need to eliminate it]**

Even though B→aa production , is not reachable to S, so it is useless variable, eliminate it also.

Keep only those production in the existing grammar, whose variable is reachable to S. do final grammar is:

S→aS|A

A→a

For example:

S→AB

A→a|B [A→a; A→B (Unit Production)]

B→b|C [B→b; B→C(unit production)]

C→aC

D→b

In the above grammar A→ a and B→b and D→ b are useless variable, rest of production not deriving any terminal, therefore it is useless, need to eliminate useless variable).

S→AB

A→a

B→b

D→b [This Production not reachable to S, therefore it need to eliminate it also]

So final grammar, after eliminating useless oroduction.

S→AB

A→a

B→b

For Example

S→AB|a

A→a|BC

B→aC|bB

C→aB|bC

D→b

Prof(Dr) Anil Kumar

In the above grammar B and C is not deriving any terminal, so B→aC|bB, C→aB|bC are useless production, it need to eliminate.
S→AB|a
A→a|BC
D→b [**This production is unreachable to S, so it is useless, eliminate it also]**
Also we need to discard B & C from above grammar
S→a
A→a
Since starting symbol, deriving terminal symbol a, so which is use A→a, discard it also.
So final simplified grammar is:
S→a

Regular Expression Vs. CFG

Difference Between Rules

- Regular and context-free grammars differ in the types of rules they allow. The rules of context-free grammars allow possible sentences as combinations of unrelated individual words (which Chomsky calls "terminals") and groups of words (phrases, or what Chomsky calls "non-terminals"). Context-free grammars allow individual words and phrases in any order and allow sentences with any number of individual words and phrases. Regular grammars, on the other hand, allow only individual words along with a single phrase per sentence. Furthermore, phrases in regular grammars must appear in the same position in every sentence or phrase, generated by the grammar.

Structures

- Because context-free grammars allow a wider range of rules than regular grammars, they can generate a wider range of structures than regular grammars. For instance, they can involve various possible structures of phrases, such as "a girl from the city with money problems" (here, the structures will vary depending on whether "with money problems" describes the city or the girl). Regular grammars cannot do this.Rather, they can generate only simple expressions consisting of strings of single, structurally independent words and possibly a single larger phrase (such as "very, very smart people").

Uses

- Context-free grammars are used in natural language processing to generate and parse language data because they can capture many of the defining features of human language, such as their potential for infinitely recursive structures. Regular grammars, which generate only a subset of the expressions of context-free grammars, are also used for natural language processing. However, they can only replicate or process short and grammatically simple linguistic expressions, such as short expressions typically found in informal dialogue

Eliminating ambiguity

If the grammar is not ambiguous then we call it unambiguous grammar. If the grammar has ambiguity then it is good for compiler construction. No method can automatically detect and remove the ambiguity, but we can remove the ambiguity by re-writing the whole grammar without ambiguity.

If RHS of more than one production starts with the same symbol,then such a grammar is called as **Grammar With Common Prefixes**.

<u>Example-</u>

$$A \rightarrow \alpha\beta_1 / \alpha\beta_2 / \alpha\beta_3$$

(Grammar with common prefixes)

- This kind of grammar creates a problematic situation for Top down parsers.
- Top down parsers can not decide which production must be chosen to parse the string in hand.

To remove this confusion, we use left factoring.

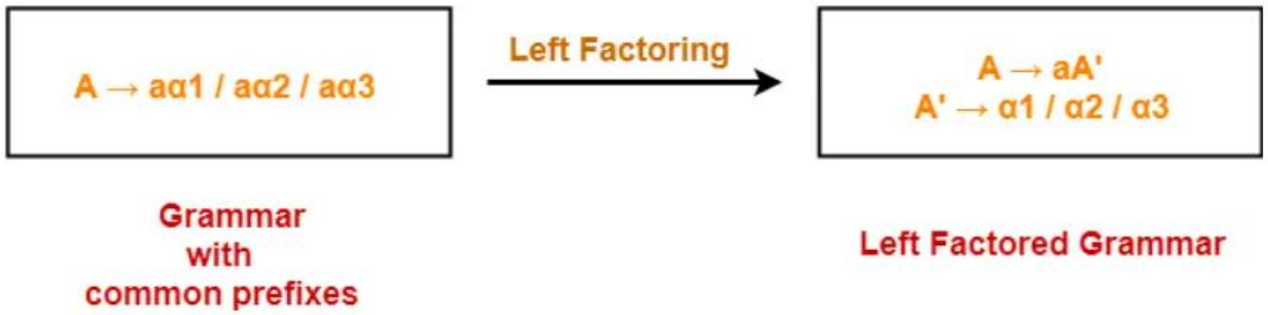
Left factoring is a process by which the grammar with common prefixes is transformed to make it useful for Top down parsers.

In left factoring,

- We make one production for each common prefixes.
- The common prefix may be a terminal or a non-terminal or a combination of both.
- Rest of the derivation is added by new productions.

The grammar obtained after the process of left factoring is called as **Left Factored Grammar**.

Example-



Problem-01:

Do left factoring in the following grammar-

$$S \rightarrow iEtS \ / \ iEtSeS \ / \ a$$

$$E \rightarrow b$$

Solution-

The left factored grammar is-

$$S \rightarrow iEtSS' \ / \ a$$

$$S' \rightarrow eS \ / \ \in$$

$$E \rightarrow b$$

Problem-02:

Do left factoring in the following grammar-

$$A \rightarrow aAB \ / \ aBc \ / \ aAc$$

Solution-

Step-01:

`` ``A → aA’

` A’ → AB / Bc / Ac

Again, this is a grammar with common prefixes.

Step-02:

$$A \rightarrow aA'$$
$$A' \rightarrow AD \ / \ Bc$$
$$D \rightarrow B \ / \ c$$

This is a left factored grammar.

Problem-03:

Do left factoring in the following grammar-

$$S \rightarrow bSSaaS \ / \ bSSaSb \ / \ bSb \ / \ a$$

Solution-

Step-01:

$$S \rightarrow bSS' \ / \ a$$
$$S' \rightarrow SaaS \ / \ SaSb \ / \ b$$

Again, this is a grammar with common prefixes.

Step-02:

$$S \rightarrow bSS' \ / \ a$$
$$S' \rightarrow SaA \ / \ b$$
$$A \rightarrow aS \ / \ Sb$$

This is a left factored grammar.

Problem-04:

Do left factoring in the following grammar-

$$S \rightarrow aSSbS \ / \ aSaSb \ / \ abb \ / \ b$$

Solution-

Step-01:

$$S \rightarrow aS' \ / \ b$$

$$S' \rightarrow SSbS \ / \ SaSb \ / \ bb$$

Again, this is a grammar with common prefixes.

Step-02:

$$S \rightarrow aS' \ / \ b$$

$$S' \rightarrow SA \ / \ bb$$

$$A \rightarrow SbS \ / \ aSb$$

This is a left factored grammar.

Problem-05:

Do left factoring in the following grammar-

$$S \rightarrow a \ / \ ab \ / \ abc \ / \ abcd$$

Solution-

Step-01:

$$S \rightarrow aS'$$

$$S' \rightarrow b \ / \ bc \ / \ bcd \ / \ \in$$

Again, this is a grammar with common prefixes.

Step-02:

$$S \rightarrow aS'$$

$$S' \rightarrow bA \,/\, \in$$

$$A \rightarrow c \,/\, cd \,/\, \in$$

Again, this is a grammar with common prefixes.

Step-03:

$$S \rightarrow aS'$$

$$S' \rightarrow bA \,/\, \in$$

$$A \rightarrow cB \,/\, \in$$

$$B \rightarrow d \,/\, \in$$

This is a left factored grammar.

Problem-06:

Do left factoring in the following grammar-

$$S \rightarrow aAd \,/\, aB$$

$$A \rightarrow a \,/\, ab$$

$$B \rightarrow ccd \,/\, ddc$$

Solution-

The left factored grammar is-

$$S \rightarrow aS'$$

$$S' \rightarrow Ad \,/\, B$$

$$A \rightarrow aA'$$

$$A' \rightarrow b \,/\, \in$$

$$B \rightarrow ccd \,/\, ddc$$

LEFT RECURSION

- A production of grammar is said to have **left recursion** if the leftmost variable of its RHS is same as variable of its LHS.
- A grammar containing a production having left recursion is called as Left Recursive Grammar.

Example-

$$S \rightarrow Sa / \in$$

(**Left Recursive Grammar**)

- Left recursion is considered to be a problematic situation for Top down parsers.
- Therefore, left recursion has to be eliminated from the grammar.

Elimination of Left Recursion

Left recursion is eliminated by converting the grammar into a right recursive grammar.

If we have the left-recursive pair of productions-

$$\mathbf{A \rightarrow A\alpha / \beta}$$

(Left Recursive Grammar)

where β does not begin with an A.

Then, we can eliminate left recursion by replacing the pair of productions with-

$$\mathbf{A \rightarrow \beta A'}$$

$$\mathbf{A' \rightarrow \alpha A' / \in}$$

(Right Recursive Grammar)

This right recursive grammar functions same as left recursive grammar.

Problem-02:

Consider the following grammar and eliminate left recursion-

$$E \rightarrow E + E / E \times E / a$$

Solution-

The grammar after eliminating left recursion is-

$E \rightarrow aA$

$A \rightarrow +EA / xEA / \in$

Problem

Consider the following grammar and eliminate left recursion-

$$E \rightarrow E + T / T$$

$$T \rightarrow T \times F / F$$

$$F \rightarrow id$$

Solution-

The grammar after eliminating left recursion is-

E → TE'

E' → +TE' / ∈

T → FT'

T' → xFT' / ∈

F → id

Problem-04:

Consider the following grammar and eliminate left recursion-

S → (L) / a

L → L , S / S

Solution-

The grammar after eliminating left recursion is-

S → (L) / a

L → SL'

L' → ,SL' / ∈

Predictive Parsing

it is top down parsing method of syntax analysis in which a set of recursive procedure us used to process the input string with procedure associated with each non-terminal grammar.
It uses an explicit stack and parsing table to do deterministic top down parsing.
Consider how we would like to parse a program in the little programming language.

- Stmt → if Stmt | Whole Stmt | Begin Stmt | ass-stmt
- If-stmt→ if Boolean then stmt else stmt
- While stmt → while bool-expr do stmt
- Begin-stmt → begin stmt-list end
- Stmt-list → stmt | stmt | stmt-list
- Bool-expr → Arith-expr compare-op arith expr
- Compare-op → <J> <=J> =J =J ! =
- **We read the Lexed Program Token-by-token from the start.**

The start symbol of the grammar is stmt.

- Suppose the first lexical class in the program is begin.
- From this information, we can tell that the first production in the syntax tree must be
- stmt → begin-stmt
- We thus have to parse the program as begin-stmt.
- We now see that the next production in the syntax tree has to be
- begin-stmt → begin stmt-list end
- We thus have to parse the full program begin …… As begin stmt-list end.

- We can thus step over begin, and proceed to parse the remaining program ……..as as stmt-list end, etc.

Algorithm

- If (X== a==s) [here TOS, X Symbol and also having $(Looking $), that mean String Successful parsed]
  - The success complete
- If ((x==a) <> $) [a----$, a is Lookahead Symbol (TOS also having a means we already derive a. so here in this situation, I want to derive next terminal, so I POP small a & increment input pointer.
  - POP TOS Symbol
  - And increment Input Pointer
- If X is Variable then T[X,a]
  - X→uvw [x→uvw is a production in Predictive Table]
  - Replace x by uvw in reverse order
- If (T,a) ==Blank then [if in Table variable X and small a is terminal, no entry, then there are parsing error]
  - ERROR

Check Grammar (LL(1)), depending upon production, 3 Rule

- First (x) = { x }, if x is a terminal
- For example: First a = {a}
- First(abc)={a} [if u are writing a string, First Terminal of String, become First]
- If x → λ or x -*-> A, then add λ to FIRST(x)
  - FIRST(x) = λ
- X→ y1, y2, y3 or x→ y1, y2, y3 ---yk when k>=1
- If grammer, production is like x→y1, y2, y3 [terminal depends on fi(y1,y2,y3]

First and Follow

First(A) gives a set of all terminals, that may begin in strings drive from A.

For example:

A→abc | def | ghr

Simplified above grammar is

A→abc [First (A) ={a

A→def [First(A) = {a, d

A→ghr [First(A) = {a, d, g}

If we want to determine First(A), First (A) is the First Terminal of all string of above production.

For Example

S→AB | b | c

A → a

Determine First of A and First of S

First (S) = {

First (A) = {a}

While we try to determine First(S), in above grammar S→AB is the first production and its first variable is non-terminal symbol so substitute its value to A→ a i.e.

S→ AB → aB

a

First (S) = {a

Next production is : S→b, First variable of this production is terminal symbol directly add in First(S) ={a ,b

Next production is : S→c, First variable of this production is terminal symbol directly add in First(S) ={a ,b, c}

First (S) = {a, b, c}

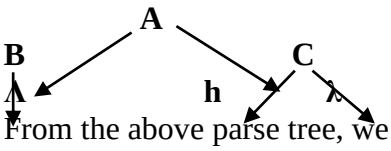
First (A) = {a}

Prof(Dr) Anil Kumar

For Example: Determine First of corresponding grammar:
S→AB
A→a
B→ b | λ
Whenever you need to find out First of any grammar, start from last production.
First(B)= { b, λ}
First(A)={a}
In case of first production S→AB, first variable is non-terminal symbol, so it need to substitute the A→a
S→AB →aB
a
So First (S)= {a}

For Example:
By using Top-down parser, determine the following grammar is LL(1) or not. if yes Justify, if not, why?
S—AaB | CbB |Ba
A→da | BC
B→ g | λ
C→ h | λ

Solution
First we need to determine or check, whether above grammar is has left recursion and left factoring.
Left recursion: if variable tends to itself, that mean grammar has left recursion. For example S→S. In above grammar there is not left recursion.
Left Factoring: is one, when grammar having repeated variable or common elements in the same production. For example A→aA | aA.
In above grammar there is no left factoring and left recursion, so can proceed further.
To verifying the above grammar whether it is LL(1) or not, need to determine First and Follow of above grammar.
First(S): of variable is terminal symbol, that came initially, whenever we are generating parse tree, when variable came starting of it what might be, first terminal symbol i.e. taken into consideration.
Start from Last Production to find out First (S)
C→h | λ [in this production 'h' is a terminal symbol and also first element of corresponding production, we can directly write either h or **λ. Here you have direct λ production, you can write directly]**
First(C) = {h, λ}
B→g | λ [in this production 'g' is a terminal symbol and also first element of corresponding production, we can directly write either h or **λ. Here you have direct λ production, we can write directly]**
First(B) = {g, λ}
A → da | BC
A→da [in this production 'd' is a terminal symbol and also first element of corresponding production, so we can write.
First(A) = {d
A→BC [here first element is non-terminal symbol i.e. B]
In above grammar one more production is there i.e. start with B, and First (B)= {g, **λ}**
First(A)={d, g,
And when we have **λ (i.e. B→ λ), during parse Tree construction, when we have BC**



From the above parse tree, we have some variable of C, i.e. First(C)= {h, **λ}**
First (A)= {d, g, h

Problem is here, both B and C has **λ,** there is possibility of B→ **λ or C→ λ** in that case what happened.
A→ **λ (A is directly derive λ), so we have to include that one also in First(A).**
First (A)= {d, g, h λ}
Next Production is S→AaB | CbB | Ba
S→AaB [in this production first of S is A(non-terminal symbol, so we have to find, what is First of A variable, i.e. First(A)={d, g, h, **λ}**
Assign to First of S:
First(S)={d, g, h [we could not assign λ, directly because there is two more production of S, (**if there is only one production, you can add λ to First(S), we know that there is one more production, so need to substitute to next element.**
S→AaB → aB [so there is possibilities that 'a' comes next elements of S]
↓
λ
First (S) = { d, g, h, a
S→CbB [in this production first of S is C(non-terminal symbol, so we have to find, what is First of C variable, i.e. First(C)={ h, **λ},**
h is already exist in First (S) = **{ d, g, h, a**
and you could not add λ because one more production is there, so substitute λ.
S→CbB →bB [So add 'b' next element of First (S)]
↓
λ

First (S) = {d, g, h, a, b
S→Ba [in this production first of S is B (non-terminal symbol, so we have to find, what is First of B variable, i.e. First(B)={ g, **λ},**
'g' is already exist in First(S)= {d, g, h, a, b
No need write 'g' in First(S);
B also having λ, so substitute B by λ

S→Ba → a [so next element of First(S) is a, but a is already exist in First (S), so No need write 'a' in First(S), Stop it.
↓
λ

First(S) ={ d, g, h, a, b}
After Find out First of Above grammar, next need to find out Follow of above Grammar.
Whenever you trying to getting Follow of grammar always start from first production.
S→AaB | CbB | Ba
A→da | BC
B→g | λ
C→h | λ
Initially, we will start $ (special symbol) for starting symbol.
S→AaB | CbB | Ba
Check expression of production, check in given definition where you have variable, S is no way, S does not occur in other place, so stop it.
Follow(S) ={$}
A→da | BC
A→da [Follow of capital A is a}
Follow (A)={a}
A→BC [**capital B occur in First Production only, i.e. S→Ba, so follow of capital B is small a, small 'a' is already exist in Follow (A), so no need to add,**

Prof(Dr) Anil Kumar

STOP it]
Follow(A)= {a}

B→g | λ [Find follow of B in whole grammar, capital B occur in 4 places, in two places capital B occur at end S→CbB and S→AaB (so when variable occur at end, then take follow of that variable.
According to this production, follow of of S be occur Follow of B, Follow(B)={$
S→Ba [**next element is a, Follow (B)= {$,a**
And in case of A→BC [Follow of B is First C, i.e. c→h | λ
Follow (B)= {$, a, h
When you have **λ, apply λ, so B to end, so we have to add Follow(A) inside,i.e. Follow(A)={a}**
'a' is already exist, in Follow (B), so no need to add and stop it.
Follow(B)={ $, a, h}

C→h | λ [in the above definition, Cis in two places, **S→CbB (according to this production Follow of C is small b, i.e. terminal symbol itself, so add it in Follow of C:**
Follow (C) = {b
A→BC [**According to this production, C comes to end, so we should add follow of A i.e. Follow (A)={a}, so add it in Follow of C**
Follow (C)= {b, a}

To verify whether grammar is LL(1) or not, need to design a Predictive Table (Predictive Parser Table).
When you fill the table, take individual production and add in table.

	A	**b**	**D**	**g**	**h**	**$**
S	**S→CbB** **S→Ba**	**S→CbB**	**S→AaB**	**S→AaB** **S→Ba**	**S→AaB** **S→CbB** **S→Ba**	**S→Ba**
A	**A→BC**		**A→da**	**A→BC**	**A→BC**	
B	**B→ λ**			**B→g**	**B→ λ**	**B→ λ**
C	**C-→λ**	**C→ λ**			**C→h**	

When you consider **C→h | λ [for λ need to check Follow, so do it later]**
You have to check First(C) = {h | λ}, First of C is First of 'h' so you need to include C→ h in Predictive table.

B→g | λ [for λ need to check Follow, so do it later]
You have to check First(B) = {g | λ}, First of B is First of 'g' so you need to include B→ g in Predictive table.
When you consider A→da | BC
A→da [First of A is First of d, i.e. First(A)= {d, so need to add A to d(A→da);
A→BC[First of A is First of B, First(A)={ d, **g, h, λ}, so add in predictive table A to g (A→BC) and A to h (A→BC).,**
Last variable is **λ, so you need to check Follow of A, i.e. Folow(A)={a}, so need to add in Predictive table A to a (A→BC).**
S→AaB | CbB | Ba
According to this production, First of S First of A, so add in Predictive table [S to a (S→AaB).
First(S)= { d, g, h, a, b}
First (A)= {d, g, h, λ**} [So add S→AaB to s to d, s to h and s to h] in last you have λ,**

Prof(Dr) Anil Kumar

Next production is S→CbB [in **this production First of S is First of C (First (C)= {h, λ}, so add c to h (S→CbB)**
This **C to h, already have one production, so you need to add one more production under it.**
In First (C)={h | λ}, next λ, need to check Follow(C) ={b,a}, so need to add in predictive table S to b (S→CbB) and S to a (S→CbB)

Next production is S→Ba [First of S is first B, First (B)={g, **λ},** so **have to include S to g (S→Ba), And you have λ, you need to check Follow of variable, Follow(B)= {$, a, h},** so need to include S to $ (S→Ba), S to a (S→Ba) and S to h (S→Ba).
Finally left out λ of every production, i.e B→ λ and C→ λ
As you already know that when you have λ you need to check Follow of variable.
Follow (B) ={ $, a, h}[so you need to include B→ **λ in predictive table(in B to $, B to a and B to h]**
Follow(C)={b, a} [so you need to include C→ **λ in predictive table(in C to b and C to a)]**
So table is completed, but problem is that, so much of places, you have two production in same column.
So above grammar is not LL(1) grammar.
For LL(1) Grammar, only single production should be in same column of Predictive Table.

Prof(Dr) Anil Kumar

LR Parser

Operator-Precedence Parser

- It is also known as Bottom-up Parsing
- We parse the tree construction from bottom to top.
- It can parse only operator Precedence grammar.
- What is Operator Precedence Grammar?
- **A grammar G, is said to be Operator Precedence, if it posses following two properties:**
- **I. No Two Production on the Right Side is ε or λ**

- **II. There should not be any production rule processing, two adjacent Non-Terminal at the Right Hand Side.**
- It is small but important class of grammar.
- Because it is used to define Mathematical Expression.
- It interprets an Operator Precedence Grammar.
- **For Example:**
- E → E A E /(E) / - E / id
- A → + / - / * / / / ^
- In this grammar, we can say that First Condition is fulfil
- But 2nd condition is not fulfil, because E A E, Here 3 non-terminal, it violet 2nd condition.
- Therefore above grammar is not Operator Precedence Grammar.
- But if replace with Production, A → + / - / * / / / ^, then we can get Operator Precedence Grammar.
- E→ E + E / E – E / E * E/ E / E / E ^ E / id
- Above grammar satisfy both condition, there is no ε RHS and no two non-terminal RHS,
- So grammar is Operator Precedence Grammar.

Design of Operator Precedence Table

- **For it, we need to design Operator Precedence Table.**
- **For it, first define three disjoint precedence relation between every pair of terminals and construct operator precedence Table.**
- **Precedence Relations**
  - **a < b a has less precedence than b**
  - **a = b a has the same precedence as b**
  - **a > b a has higher precedence than b**
- **Rule to determine Precedence Relation**
- **id has higher precedence than any other symbol**
- **$ has lowest precedence.**
- **if two operators have equal precedence, then we check the Associativity of that particular operator.**
- **Associativity: e.g. + operator is always left associativity mean those operator, towards left side solve first then RHS**

PRECEDENCE TABLE

	id	+	*	$
id		.>	.>	.>
+	<.	.>	<.	.>
*	<.	.>	.>	.>
$	<.	<.	<.	.>

Example: w= $id + id * id$

$<.id.>+<.id.>*<.id.>$

Idea Behind Operator Precedence Parsing:

- Insert precedence relations in the input string between every adjacent terminals:

- $ id_1 + id_2 * id_3 $ => $ < id_1 > + < id_2 > * < id_3 > $
- To find the handle:
  - Scan the input left to right until first >
  - scan right to left until first <
  - The handle is between < >
- In the example above: the handle is < id_1 >
- Then substitute the LHS non-terminal instead of the handle => $ E+ id_2 * id_3 $
- And so on ..
- If the string is reduced to E_1 + E_2 * E_3, we have two handles:
  - E_1 + E_2 and E_2 * E_3
- Which of them to choose?
  - Reduce the string to $ + * $ by removing non-terminals
  - Insert precedence relations: $ < + < * > $
  - The handle is: *
  - Insert non-terminals around it E_2 * E_3

For example

Construct Operator Precedence Parser for the following Grammar:

E→E A A / id

A→ + / *

Then Parse the following String

Id + id * id

Let, First we need to check, is given grammar is Operator Precedence Grammar or not

Above grammar is not a operator precedence grammar, because there are 3 non-terminal symbol together (E→E A A)

Solution

- Step1: So convert it operator precedence grammar, with help of production of A → + / *
- E→ E + E / E * E / id
- Now construct Operator Precedence Table of above grammar, check/ find out Terminal Symbol of above grammar
- Here 3 terminal symbol: + , * , id
- In addition to these terminal, we have to add one extra terminal symbol that is $.

Step2: Construct Operator Precedence Table

	id	+	*	$
id		.>	.>	.>
+	<.	.>	<.	.>
*	<.	<.	.>	.>
$	<.	<.	<.	

Prof(Dr) Anil Kumar

Parsing the given String(id + id + id)

- **Step1: Insert $ symbol at the start and end of input string($id+id+id$)**
- **Precedence operator in between every two symbols of the string by referencing the designed precedence table.**
- **Step2: start scanning, the string from left until .> and put a pointer on its location.**
- **now scan backwards the string from right to left until seeing <.**
- **Everything between the two (left angular bracket) <. And .> (right angular bracket) form the handle.**
- **Replace handle with the head of respective production**
- **Repeat the step until we reach the start.**

BASIC PRINCIPLE

- Scan input string left to right, try to detect .> and put a pointer on its location.
- Now scan backwards till reaching <.
- String between <. And .> is our handle.
- Replace handle by the head of the respective production.
- REPEAT until reaching start symbol.
- **By applying above rule- parsing the give grammar**
- **Id + id * id**

Step 1:

- Insert $ symbol start and end of input string
- Precedence operator in between every two symbol of the string by referencing the designed precedence table.

Step 2:

- Start scanning, the string from left until .> & put a pointer on its location.
- Now sacn backwards the string from right to left until <.
- Everything between the two relation (Left angular bracket) <. And .> (right angular bracket) from the handle.
- Replace handle with the Head of Respective Production
- Repeat the step, until, you reacg start symbol.

Prof(Dr) Anil Kumar

id + id * id [Given String→insert $ to start and end of string]
$id + id * id$ [insert precedence operator in between string]
$<.is.>+<.id.>*<.id.>$
[Scan the string until you get <.(right angular bracket) and then scan backward, until .>(left angular bracket), reduce it by using production.
$<.id.>+<.id.>*<.id.>$ [Replace the handle with head of respective production]
$E +<.id.>*<.id.>$ [Replace the handle with head of respective production]
$E + E * **<.id.>**$ [Replace the handle with head of respective production]
$ E + E * E$ [here you have 3 non-terminal symbol, so eliminate all 3]
$+ * $ [After eliminating, String]

$<.+ <.*.>$ [**now again insert operator, the string become]**
Again start scanning, until right angular bracket form and start scanning backward until left angular bracket found.
$<.+ <.*.>$ [it '*' which mean E * E which can be reduced E\
$<.+.>$ [**you can reduce + that mean E+E]**
$$ [after reducing you will get only start symbol, that mean you have parse the string]
For example:
Construct Operator Precedence Parser for the grammar
S→(L) | a
L→L, S | S
Also Parse the following String
(a, (a, a))
Solution:
Above grammar is operator precedence grammar, because it fulfill both properties of it. No production on right side is **λ or ε. And also** there is no adjacent non-terminal at the right hand side.
And terminal symbol in the given grammar are : { (,) , a}

Prof(Dr) Anil Kumar

First you need to build the operator precedence table for these operator.

	A	(	)	,	$
a		.>	.>	.>	.>
(	<.	.>	.>	.>	
)	<.	.>	,>	.>	.>
,	<.	<.	<.	.>	.>
$	<.	<.	<.	<.	<.

Lets start parsing the given string
(a , (a, a)) [given string\
Step 1: $(a, (a, a))$ [insert $ symbol starting and end of string]
$<.(<.a.> .<.(<.a.>, <.a.>).>).>$
Step 2: Scanning and parsing [scan the string, until you get <.(right angular bracket), then scan backward until .>(left angular bracket) and anything comes between them, it is treat as a handler & then replace it by given production. Repeat until, respective reaching production start symbol.
$<.(**<.a.>**, .<.(<.a.>, <.a.>).>).>$
$<.(S , <.(**<.a.>**, <.a.>).>).>$
$<.(**S** , <.(**S ,<.a.>**).>).>$
$<.(**S** , <.(**S ,S**).>).>$ [**consider the handle, first of all reduce S→L]**
$<.(**S** , <.(**L ,S**).>).>$ [L,S to reduce L]
$<.(**S** , <.(**L**).>).>$ [L reduce by S]
$<.(**S** , **S**).>$ [consider handle, reduced S to L]
$<.(**L** , **S**).>$
$<.(**L**).>$
$$S$$
$$ [**Finally, acceptable state, se you have parse the given string, because finally you reached to start symbol]**

LR Parser

It is bottom-up parsing technique that efficiently handles deterministic context free language in guaranteed linear time.
LR parser are used to parse the large class of CFG.
This technique is called LR(k) parsing:

- "L" is the Left-to-Right scanning of the input
- "R" is for constructing a rightmost derivation in reverse.
- K, is the number of input symbols of lookahead that is used in making parsing decision.

Principle Behind LR Parsing

- Does a right most derivation in reverse
- End with the root non-terminal on the stack
- Start with empty stack
- Uses the stack for designating what is already seen
- Build the parse tree bottom-up
- Tries to recognize a right-hand-side on the stack, pop it and pushes the corresponding non-terminals
- Reduce the non-terminals
- Read the terminals, while it pushes them on the stack
- Po st order traversal of the parse tree

WHY LR PARSING:

- LR parsers can be constructed to recognize virtually all programming-language constructs for which context-free grammars can be written.
- The LR parsing method is the most general non-backtracking **shift-reduce parsing method** known, yet it can be implemented as efficiently as other shift-reduce methods.
- The class of grammars that can be parsed using LR methods is a proper subset of the class of grammars that can be parsed with predictive parsers.
- An LR parser can detect a syntactic error as soon as it is possible to do so on a left-to-right scan of the input.

Prof(Dr) Anil Kumar

The disadvantage is that it takes too much work to constuct an LR parser by hand for a typical programming-language grammar. But there are lots of LR parser generators available to make this task easy. However, if the grammar contains ambiguities them, it is too difficult to parse in a left-to-right scan of the input.

Ambiguity

If a grammar has more than one derivation for a single sentential form, then it is ambiguous.

<stmt> :: if <expr> then < stmt >
| if <expr> then <stmt> else <stmt>
|……

Consider: if E1 and E2, then S1 and S2

- This has two derivations
- The ambiguity is purely grammatical
- It is called a CFG

Ambiguity may be eliminated by rearranging the grammar:

<stmt> :: <matched>
| <unmatched>
<matched> :: if <expr> then <matched> else
|……
<unmatched> :: if <expr> then <stmt>
| if <expr> then <matched> else <unmatched>

Ambiguity is often due to confusion in the context free specification. Confusion arise from overloading.

LR Parsing Algorithm

```
Token = next_token ( )
repeat forever
        S := Top of Stack
If action [ S, token ] = "shift Si " then
        PUSH token
        PUSH Si
        Token = next_token ( )
Elseif action [S,Token ] = "reduce A :: = ?" then
        POP 2 * |B| Symbol
        S = Top of Stack
        PUSH A
        Push goto [S,A]
Elseif action[S,token] = "accept" then
        Return
Else
        Error( )
End
```

LR(0) Items

- In case, any grammar with .(dot) create a production
- A→xyz
- We can get different production of this grammar by using .(dot)
- A→.xyz [whenever we read dot (.) RHS part invisible]
- A→x.yz [in this production x is visible, yz are invisible]
- A→xyz.

Prof(Dr) Anil Kumar

- A→ε [A→.]
- A→ wε
- A→.wε
- A→w. ε
- A→w

Example: LR(0) Items
If the production is
***E* → *E* + *T*,**
then the possible LR(0) items are
[*E* → • *E* + *T*]
[*E* → *E* • + *T*]
[*E* → *E* + • *T*]
[*E* → *E* + *T* •]
For example
Continuing with our standard example, the augmented grammar is

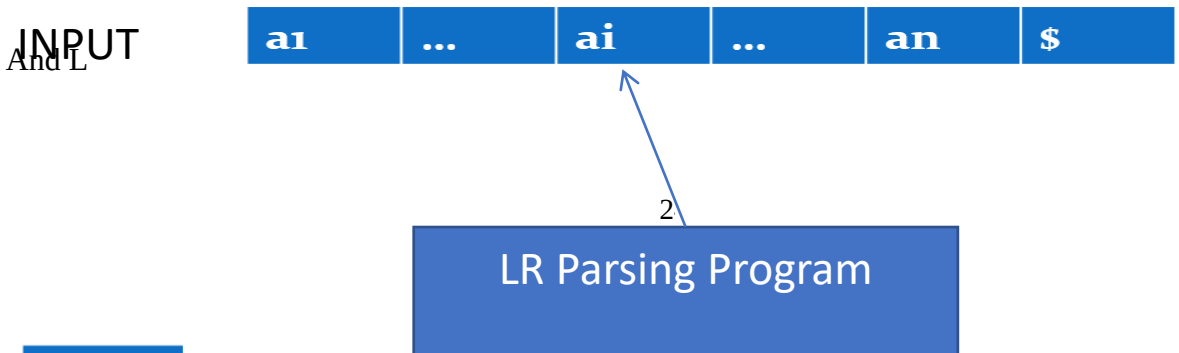
E'* → *E
E* → *E* + *T* | *T
T* → *T* * *F* | *F
***F* → (*E*) | id | num**

The state I_0 consists of the items in the closure of item [*E'* → • *E*].

[*E'* → • *E*]
[*E* → • *E* + *T*]
[*E* → • *T*]
[*T* → • *T* * *F*]
[*T* → • *F*]
[*F* → • (*E*)]
[*F* → • id]
[*F* → • num]

Transitions
There will be a transition from one state to another state for each grammar symbol in an item that immediately follows the marker • in an item in that state.
If an item in the state is [*A* → α • *X*β], then
The transition from that state occurs when the symbol *X* is processed.
The transition is to the state that is the closure of the item [*A* → α*X* • β].

LR-Parsing model:
LR Parser is an important algo. It required one input buffer, one stack and other one is LR Parsing Table.
To construct LR Parsing table, we need canonical collection of LR(0) items.



Prof(Dr) Anil Kumar

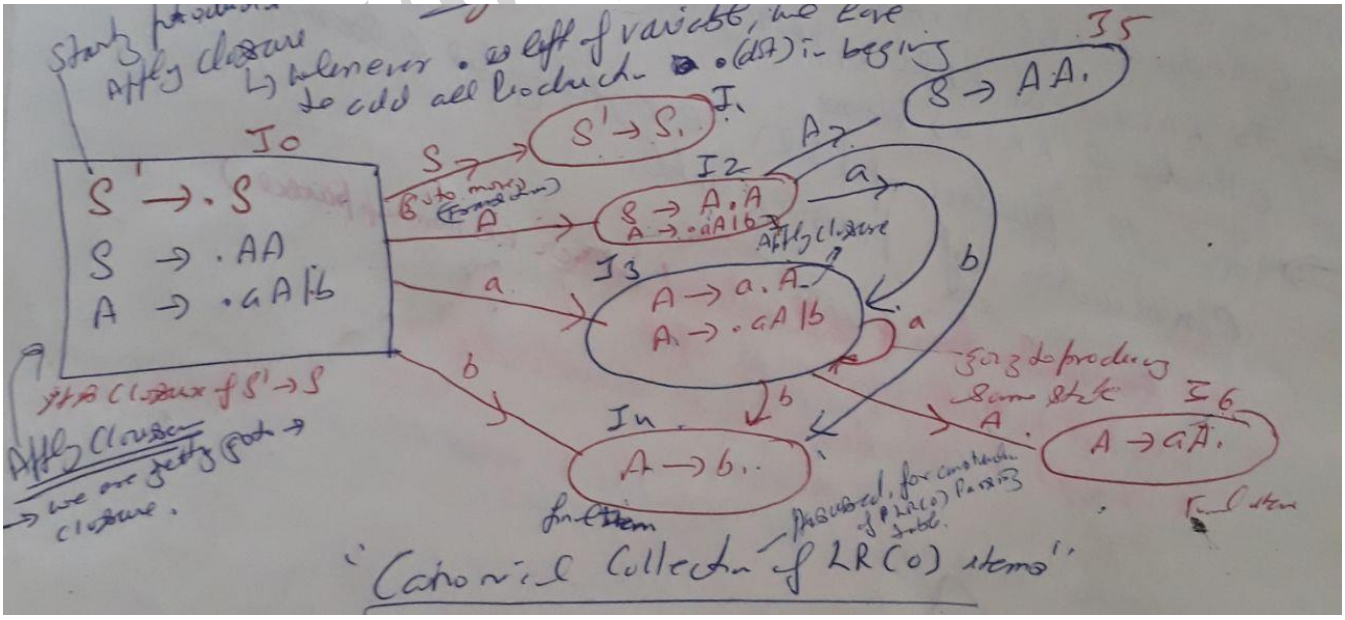
Construct canonical collection of LR(o) items and LR(0) Parsing table of following

grammar:
S→AA
A→aA | b

Like LL(1) parsing table you need to find out First and follow of given grammar, similarly in LR(0) parsing, to construct LR(0) parsing table you need closure and goto.
Whenever you want to proceed LR(0), you need to add one more production in existing grammar, i.e. S'→S to make it canonical items.

Augmented grammar

S'→S S→AA A→aA | b



Creating a transition table:

The table is index by state and symbol. we creates the states already and symbol are given by the grammar, no you need to create the action within the cells. The goto function defines the transition between closure.

- If .(dot) is at the end of an items, this is reduction action
- If symbol is a non-terminal, the action is goto
- If symbol is terminal, the action is shift

Action part and goto part common for all the parse table, only thing is different entry goes to final item, then entry will be different.

LR(0) Parse Table

State	ACTION			GOTO	
	a	b	$	A	S
0	**S3**	S4		**2**	**1**
1			**Accept**		
2	**S3**	**S4**			**5**
3	**S3**	**S4**			**6**
4	**r 3**	**r 3**	**r 3**		
5	**r 1**	**r 1**	**r 1**		
6	**r 2**	**r 2**	**r 2**		

- Number of row is the number of state
- Every shift move (S3/S4), is nothing, but whenever a state is going to terminal, i.e. Shift move
- If a state goes to variable(non-terminal), i.e. goes to goto
- And whenever, augment production (S'→S), is added as Accept move.
- And whenever, state is final item, you have to write reduced move to entire row.
- To write reduced move you need to give numbering of existing production, such as:

S→AA [1 i.e. r1]
A→aA [2 i.e. r2]
A→b [3 i.e. r3]

Prof(Dr) Anil Kumar

Shift=Reduce Parser

A shift-reduce parser is a bottom-up parsing technique that uses a stack. It shifts input symbols onto the stack and reduces them based on grammar rules until the input is completely parsed. It continuously reduces or shifts symbols until a valid parse is achieved

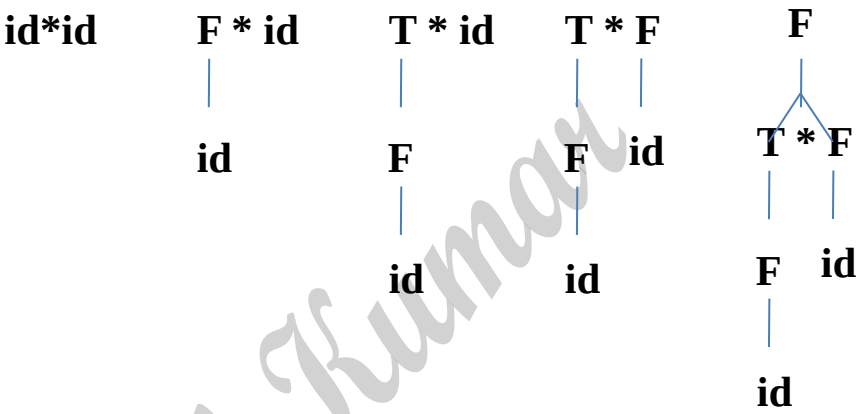
Constructs parse tree for an input string beginning at the leaves (the bottom) and working towards the root (the top)

Example: id*id.

E -> E + T | T

T -> T * F | F

F -> (E) | **id**



The general idea is to shift some symbols of input to the stack until a reduction can be applied

At each reduction step, a specific substring matching the body of a production is replaced by the nonterminal at the head of the production

The key decisions during bottom-up parsing are about when to reduce and about what production to apply

A reduction is a reverse of a step in a derivation

The goal of a bottom-up parser is to construct a derivation in reverse:

E=>T=>T*F=>T*id=>F*id=>id*id

shift Reduce parser attempts for the construction of parse in a similar manner as done in bottom-up parsing i.e. the parse tree is constructed from leaves(bottom) to the root(up). A more general form of the shift-reduce parser is the LR parser.

This parser requires some data structures i.e.

- An input buffer for storing the input string.
- A stack for storing and accessing the production rules.

Basic Operations –

- **Shift:** This involves moving symbols from the input buffer onto the stack.
- **Reduce:** If the handle appears on top of the stack then, its reduction by using appropriate production rule is done i.e. RHS of a production rule is popped out of a stack and LHS of a production rule is pushed onto the stack.
- **Accept:** If only the start symbol is present in the stack and the input buffer is empty then, the parsing action is called accept. When accepted action is obtained, it is means successful parsing is done.
- **Error:** This is the situation in which the parser can neither perform shift action nor reduce action and not even accept action.

Dr. Anil Kumar

Example 1 – Consider the grammar

S –> S + S
S –> S * S
S –> id

Perform Shift Reduce parsing for input string “id + id + id”.

Stack	Input Buffer	Parsing Action
$	id+id+id$	Shift
$id	+id+id$	Reduce S->id
$S	+id+id$	Shift
$S+	id+id$	Shift
$S+id	+id$	Reduce S->id
$S+S	+id$	Reduce S->S+S
$S	+id$	Shift
$S+	id$	Shift
$S+id	$	Reduce S->id
$S+S	$	Reduce S->S+S
$S	$	Accept

Example 2 – Consider the grammar

E –> 2E2
E –> 3E3
E –> 4

Perform Shift Reduce parsing for input string “32423”.

Stack	Input Buffer	Parsing Action
$	32423$	Shift
$3	2423$	Shift
$32	423$	Shift
$324	23$	Reduce by E --> 4
$32E	23$	Shift
$32E2	3$	Reduce by E --> 2E2
$3E	3$	Shift
$3E3	$	Reduce by E --> 3E3
$E	$	Accept

Example 3 – Consider the grammar

S –> (L) | a

L –> L , S | S

Perform Shift Reduce parsing for input string "(a, (a, a)) ".

Stack	**Input Buffer**	**Parsing Action**
$	(a , (a , a)) $	Shift
$ (	a , (a , a)) $	Shift
$ (a	, (a , a)) $	Reduce S → a
$ (S	, (a , a)) $	Reduce L → S
$ (L	, (a , a)) $	Shift
$ (L ,	(a , a)) $	Shift
$ (L , (	a , a)) $	Shift
$ (L , (a	, a)) $	Reduce S → a
$ (L , (S	, a)) $	Reduce L → S
$ (L , (L	, a)) $	Shift
$ (L , (L ,	a)) $	Shift
$ (L , (L , a	)) $	Reduce S → a
$ (L, (L, S	)) $	Reduce L →L, S
$ (L, (L	)) $	Shift
$ (L, (L)	) $	Reduce S → (L)

Prof(Dr) Anil Kumar

Stack	**Input Buffer**	**Parsing Action**
$ (L, S	) $	Reduce L → L, S
$ (L	) $	Shift
$ (L)	$	Reduce S → (L)
$ S	$	Accept

Prof(Dr) Anil Kumar